

SehatLine — Queue System Design & Algorithms

Capital Hospital, Capital Development Authority — G-6/2, Islamabad

Generated: 14/08/2026, 11:28:02 pm • How the system works, step by step

1. What the system does

SehatLine replaces the physical OPD line with a smart, priority-aware token system. A small, fixed team of doctors (3-4 chronic, 5-8 cardiology) serves 100-400 patients a day. The system decides WHO is seen first (by medical urgency and age), estimates HOW LONG each patient will wait, and spreads tokens so people are not crowding the corridor — while measuring each doctor's real pace so every estimate stays accurate on its own.

2. End-to-end workflow

- 1 Patient arrival**
Patient requests a token (chronic OPD / cardiology).
- 2 Priority assessment**
Age + condition + vitals decide the priority class (see section 4).
- 3 Priority + FCFS queue**
Higher priority first; equal priority keeps first-come-first-served.
- 4 Staff estimation (Lag SIPP)**
How many doctors are needed for the forecast demand.
- 5 M/M/s analysis (Erlang C)**
Expected wait, queue length and doctor utilisation.
- 6 Estimated wait + token**
Patient gets a token number and a realistic "~X min" wait.
- 7 Real-time updates**
Position + estimate recompute live and push to every device.

3. Algorithms used

a) M/M/s queueing model (Erlang C)

Models the OPD as s parallel doctors serving a shared queue. Inputs: arrival rate λ (patients/hour), service rate μ (patients/hour per doctor), and s doctors on duty. It computes utilisation, the chance a patient must wait, and the average wait and queue length.

$$\begin{aligned} a \text{ (offered load)} &= \lambda / \mu & \rho \text{ (utilisation)} &= a / s \\ C &= \text{Erlang-C probability an arriving patient must wait} \\ W_q \text{ (avg wait in queue)} &= C / (s * \mu - \lambda) \\ L_q \text{ (avg waiting)} &= \lambda * W_q & W &= W_q + 1/\mu & L &= \lambda * W \end{aligned}$$

b) Priority Queue

Every patient is placed in a priority class; the queue always serves the highest class first. This is what puts emergencies and the elderly ahead of standard patients (details in section 4).

c) FCFS (First-Come, First-Served)

The tie-breaker: within the SAME priority class, patients are served in arrival order, so nobody with equal urgency ever jumps the line.

d) Lag SIPP staffing

From forecast demand per time period, it computes the minimum doctors to keep the wait within target, with a "lag" shift so the busy tail of one hour stays covered by the next. This is the data-driven staffing signal (usually a 0-2 doctor or extended-hours tweak).

e) Adaptive service-rate measurement

The system records every consultation's real duration (from "doctor started" to the next stage) and continuously re-computes the average minutes-per-patient. That MEASURED pace — not a fixed guess — feeds μ , so the estimates self-tune as doctors speed up, slow down, or are added/removed.

4. How a patient's priority is decided

When a token is issued, the patient's age, condition and recent vitals are checked IN THIS ORDER — the first match wins. Higher score = seen sooner.

 Critical score 1000	Emergency/critical condition — chest pain, breathing, stroke, cardiac, kidney, unconscious, severe, etc. Served FIRST. (+100 if also elderly.)
---	---

High score 700	Pregnant, or needs special assistance/disability.
Elderly score 400	Age 50 and above — senior citizens are placed ahead of standard patients (older seniors get a small extra bump).
Normal score 100	Standard adult patient with no urgent factor.
Low score 50	Routine / non-urgent follow-ups.

Chronic illness by itself is NOT treated as an emergency (those cases go to the ED) — only true critical conditions or critical recent vitals raise a patient to the Critical class. Vitals (BP, heart rate, temperature, oxygen) can push a patient up automatically when they are dangerous.

5. How the load auto-divides across doctors

The system counts how many doctors are ON DUTY right now (the availability toggle drives this), and splits the queue across them:

```
doctors on duty  s  = count of active doctors
estimated wait   = ceil( (position - 1) / s ) x (measured minutes per patient)
```

- More doctors on duty -> each patient's "patients ahead / s" is smaller -> shorter wait.
- A doctor going off duty instantly lowers s, and every waiting patient's estimate recomputes.
- Off-duty doctors also disappear from patient booking, so load only goes to available doctors.

6. Real-time updates & self-tuning

Every token issue, "start consultation", or stage change recomputes the whole department queue in one bulk write and pushes the update over web-sockets, so positions and "~X min" estimates refresh live on the patient and doctor screens. In parallel, the measured pace refreshes every few minutes, so the same model that estimates waits also flags when the fixed doctors are genuinely over capacity (the "overloaded" signal for administration).

7. Summary

Patient arrival -> priority assessment (emergency > pregnant > elderly 50+ > normal, FCFS within a class) -> Lag SIPP staffing -> M/M/s (Erlang C) wait analysis -> token with an estimated time -> live real-time updates. The result: emergencies and seniors are seen first, normal patients get a realistic short wait, the physical crowd is removed, and the system keeps itself accurate by measuring how fast the doctors actually work.